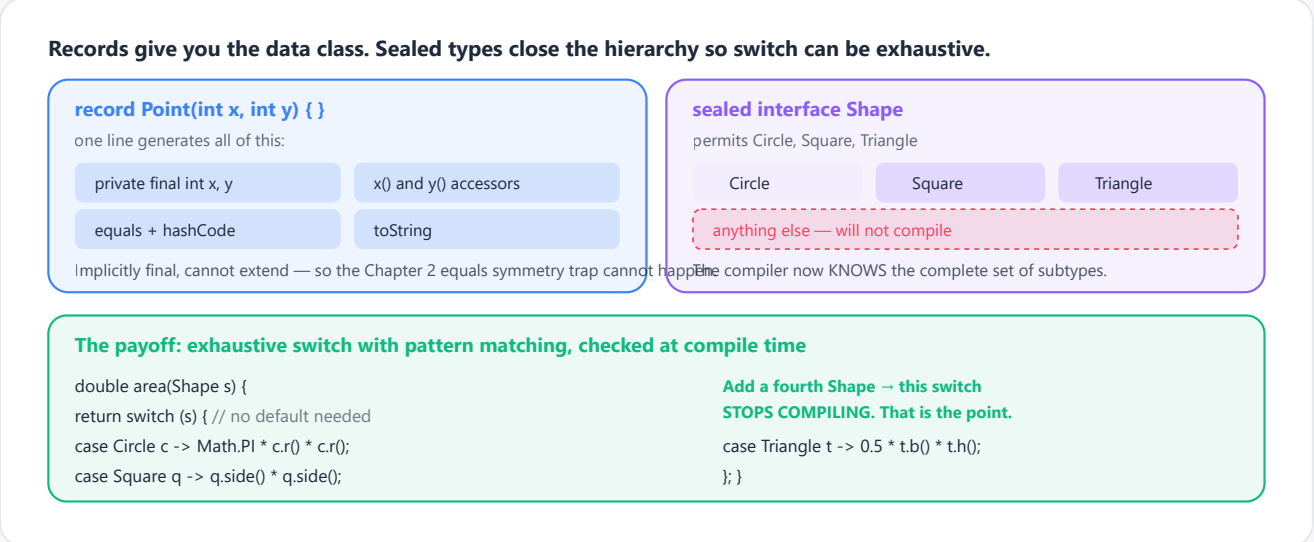
Records and Sealed Types

Why it matters

Records eliminate the single most tedious thing in Java — the data class with its constructor, accessors, `equals`, `hashCode` and `toString`. Sealed types are less famous and more interesting: they let the compiler know a hierarchy is complete, which turns a runtime "unhandled case" bug into a compile error. Together they give Java algebraic data types, and they are the modern answer to several questions from earlier chapters.

The concept, simply

A **record** (Java 16) is a transparent carrier of immutable data. A **sealed** type (Java 17) declares exactly which types may extend or implement it.



The combination is the point. A sealed hierarchy of records gives the compiler both the shape of the data and the complete set of cases, so a switch can be verified exhaustive.

How it works

What a record generates: private final fields, a canonical constructor, accessor methods named after the components (`x()` not `getX()`), plus `equals`, `hashCode` and `toString` derived from all components. Records are implicitly `final` and cannot extend a class — which neatly sidesteps the `equals` symmetry problem from Chapter 2.

Sealed types require every permitted subtype to be `final`, `sealed`, or explicitly `non-sealed` — so the hierarchy is closed at every level. Permitted subtypes must be in the same package or module.

Code examples

JAVA

Copy

```java
// The whole class:
public record Point(int x, int y) { }

// Compact constructor — validation and normalisation, no field assignment
public record Range(int low, int high) {
    public Range {                                      // no parameter list
        if (low > high) throw new IllegalArgumentException("low > high");
    }
}

// Records can have static factories, extra methods and implement interfac
public record Money(long minor, Currency currency) implements Comparable<M
    public static Money ofMajor(long major, Currency c) {
        return new Money(major * 100, c);
    }
    public Money plus(Money other) { return new Money(minor + other.minor,
    public int compareTo(Money o) { return Long.compare(minor, o.minor); }
}

// ⚠ Shallow immutability: a record component that is a mutable object is
//   still mutable. Copy defensively if it matters.
public record Team(String name, List<String> members) {
    public Team {
        members = List.copyOf(members);      // ✓ reassigning the PARAMETE
    }                                        //   in the compact construct
}
```

Sealed hierarchies, and what they buy:

JAVA

Copy

```java
public sealed interface PaymentResult
        permits Approved, Declined, Pending { }

public record Approved(String txId, Money amount) implements PaymentResult
public record Declined(String reason, String code) implements PaymentResul
public record Pending(Instant retryAfter) implements PaymentResult { }

// The compiler knows the set is complete, so no default branch is needed
// and adding a fourth case breaks this switch at COMPILE time, which is
// exactly where you want to find out.
String describe(PaymentResult r) {
    return switch (r) {
        case Approved a → "approved " + a.txId();
        case Declined d → "declined: " + d.reason();
        case Pending p  → "retry after " + p.retryAfter();
    };
}

// Compare with an enum + fields, the old way of modelling this: an enum
// cannot carry different data per case, so you end up with nullable field
// that are only meaningful for some values.
```

Where records are the wrong tool:

JAVA

Copy

```java
// ✗ Not for JPA entities. They need a no-arg constructor, mutable fields
//   non-final classes for proxying. Records give none of those.
// ✗ Not when you need inheritance — records are implicitly final.
// ✗ Not when fields must change after construction.
// ✓ Ideal for: DTOs, API responses, value objects, method return tuples,
//   map keys (correct equals/hashCode by construction), and the leaves of
//   a sealed hierarchy.
```

Under the hood

implemented via `invokedynamic` into `ObjectMethods.bootstrap` rather than being spelled out in bytecode, which keeps class files small and lets the implementation improve without recompilation.

Records are shallowly immutable, and this is the caveat that matters in review. The *reference* in each component is final; the object it points at may be perfectly mutable. A record holding an `ArrayList` hands out that list to every caller. The fix is a defensive copy in the compact constructor — which works because the compact constructor assigns the parameters to the fields *after* your code runs, so reassigning the parameter is the idiom.

Sealed types exist to make exhaustiveness checkable. Without them the compiler must assume any type could implement an interface, so it demands a `default` branch — and a `default` silently absorbs the new case you forgot to handle. Sealing turns "I hope I updated every switch" into a build failure, which is the entire value proposition.

Common mistakes

- **Expecting deep immutability.** Mutable components stay mutable — copy in the compact constructor.
- **Using records as JPA entities.** No no-arg constructor, no mutability, final class — incompatible.
- **Assigning fields in a compact constructor.** Reassign the *parameter*; the field assignment is generated.
- **Expecting `getX()`.** Accessors are named `x()`, which some older frameworks do not recognise.
- **Forgetting permitted subtypes must be `final`, `sealed` or `non-sealed`.**
- **Adding a `default` branch to a sealed switch.** It defeats exhaustiveness checking — the whole point.

Interview lens

A: A transparent carrier for immutable data. From the component list the compiler generates private final fields, a canonical constructor, accessors named after the components, and `equals`, `hashCode` and `toString` derived from all of them. Records are implicitly final and cannot extend a class, which is what makes their `equals` contract safe — the subclass symmetry problem cannot arise.

Q: "Are records immutable?"

A: Shallowly. The component fields are final, but if a component is a mutable object — a `List`, a `Date` — that object can still be modified by anyone holding it. For real immutability you copy defensively in the compact constructor, reassigning the parameter, since the field assignment is generated after your code runs.

Q: "What problem do sealed types solve?"

A: They tell the compiler the complete set of subtypes, which makes exhaustiveness checkable. Without sealing, a switch over an interface needs a `default` branch, and that branch silently swallows any new implementation you forgot to handle. With a sealed hierarchy, adding a case breaks every switch that does not handle it — at compile time, which is where you want to discover it.

Q: "When would you not use a record?"

A: JPA entities, because they need a no-arg constructor, mutable fields and a non-final class for proxying. Anything requiring inheritance, since records are final. And anything whose state must change after construction. For DTOs, API responses, value objects and map keys they are close to ideal.

Q: "Records and sealed interfaces together — why?"

A: They give Java algebraic data types. The sealed interface defines the closed set of alternatives, and each record defines the data that case carries — which an enum cannot do, since every enum constant shares one shape. Combined with

pattern-matching switch you get a total function over the type, verified by the compiler.

Cheat-card

- Record generates: **final fields · canonical constructor · `x()` accessors · equals/hashCode/toString**.
- Implicitly **final**, extends `java.lang.Record`, cannot extend a class.
- **Shallowly immutable** — defensive-copy mutable components in the **compact constructor** (reassign the parameter).
- Not for **JPA entities**; ideal for DTOs, value objects, map keys, sealed leaves.
- `sealed … permits`; subtypes must be `final`, `sealed` or `non-sealed`, same package/module.
- Sealed ⇒ **exhaustive switch, no `default`** ⇒ adding a case is a compile error.
- Sealed interface + records = **algebraic data types**, which enums cannot express.

QUICK CHECK — no marks, just for you.

1. A record has a List component. Is the record immutable?

- ◯ Only if the record is declared final
- ◯ Only shallowly — the field reference is final, but the List it points at can still be modified by anyone holding it
- ◯ Yes, fully — records deep-copy all components
- ◯ No, records are always mutable

Check answer

2. Why can a record not be a JPA entity?

- Records cannot implement interfaces
- JPA needs a no-arg constructor, mutable fields to populate, and a non-final class for lazy proxies — records provide none of those
- Records cannot have annotations

Check answer

3. What do sealed types buy you that a plain interface does not?

- Protection against reflection
- Better run-time performance
- Automatic serialisation
- The compiler knows the complete set of subtypes, so a switch over them can be verified EXHAUSTIVE — adding a case breaks the build instead of silently falling into default

Check answer

‹ Previous
var, Text Blocks and Small Language Wins

Next ›
Pattern Matching and Switch Expressions

LEARN MORE

ARTICLES & DOCS

Records (JEP 395)

openjdk.org

Sealed classes (JEP 409)

openjdk.org

Record — API docs

docs.oracle.com

VIDEOS



Java Records, Sealed Classes & Pattern Matching — Modern Java Explained

Ebi



Clean Code with Records, Sealed Classes and Pattern Matching by José Paumard

Devoxx

Hand-picked references for this lesson.